Weekly Report - Week 4

Semester 2, 2025 | Date: August 24, 2025
Student: Yongchun Li (1378156) | **Epic:** 5.1 Booking and Scheduling Foundation

---

1. Work Completed

This week I implemented **Epic 5.1 (Booking and Scheduling System Foundation)** for OAS, establishing core infrastructure for assessment scheduling and booking functionality supporting user stories 5.1.1-5.1.3.

Code Branch: Epic5.1-Dev

1.1 Database Schema Design (4h)

Implemented three-layer database architecture supporting flexible assessment scheduling:

- `assessment_session_config`: Core session parameters, examiner allocation rules, and booking policies
- `assessment_period`: Time block management with venue information and template duplication
- `assessment_slot`: Individual bookable slots with status tracking and reschedule history
- `examiner_slot_assignment`: Self-assignment tracking with credit calculation
- `assessment_booking_history`: Complete audit trail for booking changes

Key Design Features:

- Service-layer logical relationships (no DB foreign keys) for flexibility
- JSON-based examiner allocation structure for dynamic credit management
- Template-based period duplication for recurring assessments
- Comprehensive status tracking (AVAILABLE, BOOKED, COMPLETED, CANCELLED, BLOCKED)

1.2 Entity & Service Implementation (7h)

Entity Layer:

- Complete JPA entities with MyBatis-Plus integration and Swagger documentation
- Lombok integration for clean code structure and validation enforcement

Service Layer:

- **AssessmentSessionConfigService**: Configuration management with validation
- **AssessmentPeriodService**: Period CRUD with template duplication and batch creation
- **AssessmentSlotService**: Slot management with automatic generation
- **AssessmentBookingService**: Student booking operations with conflict detection

Key Features:

- Automatic slot generation based on configuration parameters
- Comprehensive booking validation and time conflict detection

- Template-based duplication with customizable parameters
- Advanced reschedule policy enforcement with deadline validation

1.3 DTO Design & Utilities (3h)

Data Transfer Objects:

- **AssessmentSessionConfigCreateDTO**: Configuration creation with validation
- **AssessmentPeriodCreateDTO/ResponseDTO**: Period management with nested information
- **StudentBookingRequestDTO**: Booking requests with type validation (INDIVIDUAL/GROUP)
- **ExaminerAssignmentRequestDTO**: Self-assignment requests with availability checking

SchedulingUtils Framework:

- Booking time window validation and enforcement
- Reschedule deadline calculation and validation
- Time conflict detection algorithms
- Slot capacity and availability checking utilities

---

2. Technical Achievements

2.1 Database Innovation

- **Three-layer architecture** supporting complex scheduling scenarios
- **JSON-based configuration** enabling dynamic examiner allocation and credit management
- **Template system** for efficient period duplication and reuse

2.2 Business Logic Excellence

- **Automatic slot generation** algorithm considering duration, gaps, and venue constraints
- **Intelligent conflict detection** preventing double-booking scenarios
- **Flexible reschedule policies** with configurable deadline enforcement

2.3 Code Quality Standards

- Consistent error handling using established `BusinessException` patterns
- Complete Swagger documentation for API self-documentation
- Comprehensive validation at DTO and service layers with transaction management

---

3. Key Challenges & Solutions

Challenge	Solution
Complex Entity Relationships	Service-layer validation replacing database foreign keys

Challenge	Solution
Dynamic Examiner Allocation	Extensible JSON format: `[{"userId":1,"credits":10.5}...]`
Slot Generation Algorithm	Configurable algorithm supporting various scheduling patterns

---

4. Next Week's Plan

4.1 Help with Epic 5.2-5.3 Implementation

- Student booking interface and validation logic
- Examiner self-assignment workflow with real-time availability checking

4.2 System Integration

- Frontend-backend API integration and testing
- Email notification system for booking confirmations
- Performance optimization for large-scale scenarios

Total Effort: ~14 hours | **Status:** Epic 5.1 foundation almost complete